

HireIQ

AI Hiring Intelligence Platform

Backend Specification Document (BSD)

Document Type	Backend Specification Document (BSD)
Product	HireIQ — AI Hiring Intelligence Platform
Version	v1.0.0
Parent Documents	HireIQ PRD v1.0 HireIQ TAD v1.0 HireIQ SACD v1.0
Status	DRAFT — Backend Engineering Review
Author	Backend Engineering Team
Date	June 14, 2026
Framework	FastAPI (Python 3.11+) + PostgreSQL + Redis + AWS S3

SECTION 1

Backend Overview & Service Architecture

1. Backend Overview & Service Architecture

This document provides the complete backend implementation specification for HireIQ. It covers the FastAPI application structure, all route implementations, service layer business logic, database query patterns, middleware chain, error handling, caching strategy, and testing requirements. Engineers must follow these specifications exactly to ensure consistency, security, and maintainability.

1.1 Service Boundaries

Core API Service	Port 8000. Handles all primary business logic: authentication, user management, job CRUD, resume processing, application management, and recruiter analytics. Connects to PostgreSQL, Redis, and S3.
AI / ML Service	Port 8001. Stateless inference service. Handles resume parsing, candidate scoring, job recommendations, and interview evaluation. Receives data via HTTP from Core API. Returns JSON results only.
Database (PostgreSQL)	Port 5432. Primary data store. Accessed only by Core API service via SQLAlchemy ORM. Never accessed directly from frontend or AI service.
Cache (Redis)	Port 6379. Shared between Core API and AI Service. Used for caching query results, AI scores, rate limiting counters, and refresh token storage.
File Storage (AWS S3)	Accessed via boto3 SDK from Core API only. Frontend uploads directly to S3 via pre-signed URLs. No binary file data passes through the API server.

1.2 FastAPI Application Factory

```
# main.py - FastAPI application factory with all configuration
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from contextlib import asynccontextmanager
from app.core.database import engine, Base
from app.core.redis import init_redis
from app.api.v1.routes import auth, users, jobs, resumes, applications, ai, recruiter
from app.middlewares.logging_middleware import LoggingMiddleware
from app.middlewares.rate_limit import RateLimitMiddleware

@asynccontextmanager
async def lifespan(app: FastAPI):
```

```

# Startup: initialise DB pool and Redis connection
await init_redis()
async with engine.begin() as conn:
    await conn.run_sync(Base.metadata.create_all)
yield
# Shutdown: close connections gracefully
await engine.dispose()

app = FastAPI(
    title='HireIQ API',
    version='1.0.0',
    docs_url='/docs',          # Swagger UI (disable in production)
    redoc_url='/redoc',
    lifespan=lifespan
)

# CORS - allow only the frontend origin
app.add_middleware(CORSMiddleware,
    allow_origins=['https://hireiq.io', 'http://localhost:3000'],
    allow_credentials=True,    # Required for httpOnly cookie refresh token
    allow_methods=['*'],
    allow_headers=['*'],
)

app.add_middleware(LoggingMiddleware) # Structured JSON request logging
app.add_middleware(RateLimitMiddleware) # Redis-backed rate limiting

# Register all route groups
app.include_router(auth.router, prefix='/v1/auth',
    tags=['Authentication'])
app.include_router(users.router, prefix='/v1/users', tags=['Users'])
app.include_router(jobs.router, prefix='/v1/jobs', tags=['Jobs'])
app.include_router(resumes.router, prefix='/v1/resumes', tags=['Resumes'])
app.include_router(applications.router, prefix='/v1/applications', tags=['Applications'])
app.include_router(ai.router, prefix='/v1/ai', tags=['AI'])
app.include_router(recruiter.router, prefix='/v1/recruiter', tags=['Recruiter'])

@app.get('/health')
async def health_check():
    return {'status': 'ok', 'version': '1.0.0'}

```

SECTION 2

Database Layer — Models & Sessions

2. Database Layer — SQLAlchemy Models & Sessions

2.1 Database Configuration

```
# app/core/database.py
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession,
    async_sessionmaker
from sqlalchemy.orm import DeclarativeBase
from app.core.config import settings

engine = create_async_engine(
    settings.DATABASE_URL,
    pool_size=settings.DB_POOL_MIN,      # Min connections in pool (default: 5)
    max_overflow=settings.DB_POOL_MAX,    # Max overflow connections (default: 15)
    pool_pre_ping=True,                  # Test connection before checkout (handles stale
connections)
    pool_recycle=3600,                   # Recycle connections every hour
    echo=settings.DEBUG,                 # Log SQL queries in debug mode only
)

AsyncSessionLocal = async_sessionmaker(
    bind=engine,
    class_=AsyncSession,
    expire_on_commit=False,              # Keep objects usable after commit (no lazy load
issues)
)

class Base(DeclarativeBase):
    pass

# Dependency injected into every route that needs DB access
async def get_db() -> AsyncSession:
    async with AsyncSessionLocal() as session:
        try:
            yield session
            await session.commit()
        except Exception:
            await session.rollback()
            raise
        finally:
            await session.close()
```

2.2 SQLAlchemy ORM Models

```

# app/models/user.py
from sqlalchemy import Column, String, Boolean, Integer, ARRAY, Text
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.sql import func
from sqlalchemy.orm import relationship
import uuid
from app.core.database import Base

class User(Base):
    __tablename__ = 'users'

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    name = Column(String(255), nullable=False)
    email = Column(String(255), unique=True, nullable=False, index=True)
    password_hash = Column(String(255), nullable=False)
    role = Column(String(20), nullable=False) # job_seeker | recruiter | admin
    bio = Column(Text, nullable=True)
    skills = Column(ARRAY(String), server_default='{}')
    experience = Column(String(100), nullable=True)
    education = Column(String(255), nullable=True)
    avatar_url = Column(String(500), nullable=True)
    profile_pct = Column(Integer, default=0)
    is_active = Column(Boolean, default=True)
    created_at = Column(TIMESTAMPZ, server_default=func.now())
    updated_at = Column(TIMESTAMPZ, server_default=func.now(),
onupdate=func.now())

    # Relationships
    jobs = relationship('Job', back_populates='recruiter', lazy='select')
    resume = relationship('Resume', back_populates='user', uselist=False)
    applications = relationship('Application', back_populates='user')

# app/models/job.py
class Job(Base):
    __tablename__ = 'jobs'

    id = Column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    title = Column(String(255), nullable=False)
    description = Column(Text, nullable=False)
    required_skills = Column(ARRAY(String), server_default='{}')
    salary_min = Column(Integer, nullable=True)
    salary_max = Column(Integer, nullable=True)
    location = Column(String(255), nullable=True)
    job_type = Column(String(50), nullable=True)
    status = Column(String(20), default='active')
    recruiter_id = Column(UUID(as_uuid=True), ForeignKey('users.id',
ondelete='CASCADE'))
    applicant_count = Column(Integer, default=0)
    created_at = Column(TIMESTAMPZ, server_default=func.now())
    updated_at = Column(TIMESTAMPZ, server_default=func.now(),
onupdate=func.now())

    recruiter = relationship('User', back_populates='jobs')
    applications = relationship('Application', back_populates='job')

```

2.3 Pydantic Schema Patterns

```
# app/schemas/job.py - Request / Response schemas
from pydantic import BaseModel, Field, field_validator, ConfigDict
from typing import Optional, List
from uuid import UUID
from datetime import datetime

# Base schema with shared fields
class JobBase(BaseModel):
    title: str = Field(min_length=3, max_length=255)
    description: str = Field(min_length=50, max_length=10000)
    required_skills: List[str] = Field(min_length=1, max_length=20)
    salary_min: Optional[int] = Field(None, ge=0)
    salary_max: Optional[int] = Field(None, ge=0)
    location: Optional[str] = Field(None, max_length=255)
    job_type: Optional[str] = None

# Create request - used in POST /jobs
class JobCreate(JobBase):
    model_config = ConfigDict(strict=True, str_strip_whitespace=True)

    @field_validator('required_skills')
    @classmethod
    def clean_skills(cls, v): return [s.strip().lower() for s in v if s.strip()]

# Update request - used in PUT /jobs/:id (all fields optional)
class JobUpdate(BaseModel):
    title: Optional[str] = None
    description: Optional[str] = None
    required_skills: Optional[List[str]] = None
    salary_min: Optional[int] = None
    salary_max: Optional[int] = None
    location: Optional[str] = None
    job_type: Optional[str] = None

# Response schema - returned from GET /jobs and GET /jobs/:id
class JobResponse(JobBase):
    id: UUID
    status: str
    recruiter_id: UUID
    applicant_count: int
    created_at: datetime
    updated_at: datetime
    match_score: Optional[int] = None # Populated for job seeker context

    model_config = ConfigDict(from_attributes=True) # Allows ORM model input

# Paginated response wrapper
class PaginatedJobs(BaseModel):
    data: List[JobResponse]
    total: int
    page: int
```

```
pages:    int
has_next: bool
```

SECTION 3

Authentication Module Implementation

3. Authentication Module

3.1 Security Utilities

```
# app/core/security.py - JWT and password utilities
from datetime import datetime, timedelta, timezone
from jose import jwt, JWTError
from passlib.context import CryptContext
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from sqlalchemy.ext.asyncio import AsyncSession
from app.core.config import settings
from app.core.database import get_db
from app.models.user import User

# bcrypt context with cost factor 12
pwd_context = CryptContext(schemes=['bcrypt'], deprecated='auto',
    bcrypt__rounds=12)
bearer_scheme = HTTPBearer()

def hash_password(password: str) -> str:
    return pwd_context.hash(password) # ~250ms intentional delay

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)

def create_access_token(user_id: str, role: str) -> str:
    payload = {
        'sub': str(user_id),
        'role': role,
        'iat': datetime.now(timezone.utc),
        'exp': datetime.now(timezone.utc) +
            timedelta(minutes=settings.JWT_ACCESS_TOKEN_EXPIRE_MINUTES)
    }
    return jwt.encode(payload, settings.JWT_SECRET_KEY,
        algorithm=settings.JWT_ALGORITHM)

# Dependency: extracts and validates JWT from Authorization header
async def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(bearer_scheme),
    db: AsyncSession = Depends(get_db)
) -> User:
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail={'error': 'Invalid or expired token', 'code': 'INVALID_TOKEN'},
```



```

        headers={'WWW-Authenticate': 'Bearer'},
    )
    try:
        payload = jwt.decode(credentials.credentials, settings.JWT_SECRET_KEY,
                               algorithms=[settings.JWT_ALGORITHM])
        user_id: str = payload.get('sub')
        if user_id is None: raise credentials_exception
    except JWTError:
        raise credentials_exception

    user = await db.get(User, user_id)
    if user is None or not user.is_active:
        raise credentials_exception
    return user

```

3.2 Auth Routes Implementation

```

# app/api/v1/routes/auth.py
from fastapi import APIRouter, Depends, HTTPException, Response, Cookie
from sqlalchemy.ext.asyncio import AsyncSession
from app.core.database import get_db
from app.core.security import hash_password, verify_password, create_access_token
from app.core.redis import redis_client
from app.models.user import User
from app.schemas.auth import RegisterRequest, LoginRequest, TokenResponse
from app.services.auth_service import AuthService
import uuid

router = APIRouter()

@router.post('/register', response_model=TokenResponse, status_code=201)
async def register(body: RegisterRequest, response: Response, db: AsyncSession =
Depends(get_db)):
    # Check duplicate email
    existing = await db.execute(select(User).where(User.email == body.email))
    if existing.scalar_one_or_none():
        raise HTTPException(409, detail={'error': 'Email already registered',
'code': 'EMAIL_EXISTS'})

    user = User(
        name=body.name, email=body.email,
        password_hash=hash_password(body.password),
        role=body.role
    )
    db.add(user)
    await db.flush() # Get user.id without full commit

    access_token = create_access_token(user.id, user.role)
    refresh_token = str(uuid.uuid4())

    # Store hashed refresh token in Redis with 30-day expiry
    await redis_client.setex(f'refresh:{user.id}', 86400 * 30, refresh_token)

```

```

# Set refresh token as httpOnly secure cookie
response.set_cookie('refresh_token', refresh_token,
    httponly=True, secure=True, samesite='lax', max_age=86400*30)

await db.commit()
return {'access_token': access_token, 'token_type': 'bearer',
    'user': UserResponse.model_validate(user)}

@router.post('/login', response_model=TokenResponse)
async def login(body: LoginRequest, response: Response, db: AsyncSession =
Depends(get_db)):
    result = await db.execute(select(User).where(User.email == body.email))
    user = result.scalar_one_or_none()

    # Always run verify_password even if user not found (timing attack prevention)
    dummy_hash = '$2b$12$dummy.hash.to.prevent.timing.attack.on.user.enumeration'
    stored_hash = user.password_hash if user else dummy_hash

    if not verify_password(body.password, stored_hash) or not user:
        await AuthService.record_failed_attempt(body.email) # Rate limit tracking
        raise HTTPException(401, detail={'error': 'Invalid credentials', 'code':
'INVALID_CREDENTIALS'})

    access_token = create_access_token(user.id, user.role)
    refresh_token = str(uuid.uuid4())
    await redis_client.setex(f'refresh:{user.id}', 86400 * 30, refresh_token)
    response.set_cookie('refresh_token', refresh_token,
        httponly=True, secure=True, samesite='lax', max_age=86400*30)

    return {'access_token': access_token, 'token_type': 'bearer',
        'user': UserResponse.model_validate(user)}

```

SECTION 4

Jobs Module Implementation

4. Jobs Module Implementation

4.1 Job Service — Business Logic

```
# app/services/job_service.py
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select, func, text, or_, and_
from sqlalchemy.dialects.postgresql import array
from app.models.job import Job
from app.schemas.job import JobCreate, JobUpdate, PaginatedJobs
from app.core.redis import redis_client
from app.utils.pagination import paginate
import json, hashlib

class JobService:

    @staticmethod
    async def get_jobs(db: AsyncSession, page: int, limit: int,
                      search: str | None, skills: list[str] | None,
                      location: str | None, salary_min: int | None,
                      salary_max: int | None, sort_by: str = 'created_at') ->
PaginatedJobs:

        # Generate cache key from query params
        cache_key = f'job_list:{hashlib.md5(str(locals()).encode()).hexdigest()}'
        cached = await redis_client.get(cache_key)
        if cached: return PaginatedJobs(**json.loads(cached))

        query = select(Job).where(Job.status == 'active')

        # Full-text search across title and description
        if search:
            query = query.where(
                text('to_tsvector(english, title || description) @@
plainto_tsquery(english, q)'),
                .bindparams(q=search)
            )

        # Skills filter - job must have ALL requested skills
        if skills:
            query = query.where(Job.required_skills.contains(skills))

        # Location filter (case-insensitive partial match)
        if location:
            query = query.where(Job.location.ilike(f'%{location}%'))
```

```

        # Salary range filter
        if salary_min is not None:
            query = query.where(Job.salary_max >= salary_min)
        if salary_max is not None:
            query = query.where(Job.salary_min <= salary_max)

        # Sorting
        sort_map = {'created_at': Job.created_at.desc(),
                    'salary': Job.salary_max.desc(),
                    'applicants': Job.applicant_count.desc()}
        query = query.order_by(sort_map.get(sort_by, Job.created_at.desc()))

        result = await paginate(db, query, page, limit)

        # Cache for 5 minutes
        await redis_client.setex(cache_key, 300, json.dumps(result.model_dump()))
        return result

    @staticmethod
    async def create_job(db: AsyncSession, data: JobCreate, recruiter_id: str) -> Job:
        job = Job(**data.model_dump(), recruiter_id=recruiter_id)
        db.add(job)
        await db.commit()
        await db.refresh(job)
        # Invalidate job listing cache on new job creation
        await redis_client.delete_pattern('job_list:*')
        return job

    @staticmethod
    async def verify_ownership(db: AsyncSession, job_id: str, user_id: str) -> Job:
        job = await db.get(Job, job_id)
        if not job:
            raise HTTPException(404, detail={'error': 'Job not found', 'code': 'JOB_NOT_FOUND'})
        if str(job.recruiter_id) != str(user_id):
            raise HTTPException(403, detail={'error': 'You do not own this job', 'code': 'NOT_JOB_OWNER'})
        return job

```

4.2 Jobs Route Handler

```

# app/api/v1/routes/jobs.py
from fastapi import APIRouter, Depends, Query
from sqlalchemy.ext.asyncio import AsyncSession
from app.core.database import get_db
from app.core.security import get_current_user
from app.middlewares.role_middleware import require_role
from app.models.user import User
from app.schemas.job import JobCreate, JobUpdate, JobResponse, PaginatedJobs
from app.services.job_service import JobService

```

```

router = APIRouter()

@router.get('/', response_model=PaginatedJobs)
async def list_jobs(
    page: int = Query(1, ge=1),
    limit: int = Query(10, ge=1, le=50),
    search: str | None = Query(None, max_length=200),
    skills: list[str] | None = Query(None),
    location: str | None = Query(None),
    salary_min: int | None = Query(None, ge=0),
    salary_max: int | None = Query(None, ge=0),
    sort_by: str = Query('created_at'),
    db: AsyncSession = Depends(get_db),
    _: User = Depends(get_current_user) # Auth required
):
    return await JobService.get_jobs(db, page, limit, search, skills,
                                     location, salary_min, salary_max, sort_by)

@router.post('/', response_model=JobResponse, status_code=201)
@require_role('recruiter', 'admin')
async def create_job(
    body: JobCreate = ...,
    db: AsyncSession = Depends(get_db),
    current_user: User = Depends(get_current_user)
):
    return await JobService.create_job(db, body, current_user.id)

@router.get('/{job_id}', response_model=JobResponse)
async def get_job(
    job_id: str,
    db: AsyncSession = Depends(get_db),
    _: User = Depends(get_current_user)
):
    job = await db.get(Job, job_id)
    if not job: raise HTTPException(404, detail={'error': 'Job not found'})
    return job

@router.put('/{job_id}', response_model=JobResponse)
@require_role('recruiter', 'admin')
async def update_job(
    job_id: str,
    body: JobUpdate = ...,
    db: AsyncSession = Depends(get_db),
    current_user: User = Depends(get_current_user)
):
    job = await JobService.verify_ownership(db, job_id, current_user.id)
    for field, value in body.model_dump(exclude_none=True).items():
        setattr(job, field, value)
    await db.commit()
    await db.refresh(job)
    await redis_client.delete_pattern('job_list:*')
    return job

@router.delete('/{job_id}', status_code=204)
@require_role('recruiter', 'admin')
async def delete_job(

```

```
    job_id: str, db: AsyncSession = Depends(get_db),
    current_user: User = Depends(get_current_user)
):
    job = await JobService.verify_ownership(db, job_id, current_user.id)
    await db.delete(job)
    await db.commit()
    await redis_client.delete_pattern('job_list:*')
```

SECTION 5

Resume Module Implementation

5. Resume Module Implementation

5.1 S3 Upload Utility

```
# app/utils/s3.py - S3 operations via boto3
import boto3
from botocore.exceptions import ClientError
from app.core.config import settings
from uuid import uuid4

s3_client = boto3.client(
    's3',
    aws_access_key_id = settings.AWS_ACCESS_KEY_ID,
    aws_secret_access_key = settings.AWS_SECRET_ACCESS_KEY,
    region_name = settings.AWS_REGION
)

def generate_presigned_upload_url(user_id: str) -> tuple[str, str]:
    key = f'resumes/{user_id}/{uuid4()}.pdf'
    url = s3_client.generate_presigned_url(
        'put_object',
        Params={
            'Bucket': settings.S3_BUCKET_NAME,
            'Key': key,
            'ContentType': 'application/pdf',
        },
        ExpiresIn=900 # 15 minutes
    )
    return url,
f'https://{settings.S3_BUCKET_NAME}.s3.{settings.AWS_REGION}.amazonaws.com/{key}'

async def delete_s3_object(file_url: str) -> None:
    key = file_url.split('.amazonaws.com/')[1]
    try:
        s3_client.delete_object(Bucket=settings.S3_BUCKET_NAME, Key=key)
    except ClientError as e:
        logger.error(f'S3 delete failed: {e}', extra={'key': key})
        raise HTTPException(500, detail={'error': 'Failed to delete file from storage'})

def generate_presigned_view_url(file_url: str, expiry: int = 3600) -> str:
    key = file_url.split('.amazonaws.com/')[1]
    return s3_client.generate_presigned_url(
        'get_object',
        Params={'Bucket': settings.S3_BUCKET_NAME, 'Key': key},
        ExpiresIn=expiry
    )
```

```
)
```

5.2 Resume Service

```
# app/services/resume_service.py
from fastapi import UploadFile, HTTPException, BackgroundTasks
from sqlalchemy.ext.asyncio import AsyncSession
from app.models.resume import Resume
from app.utils.s3 import delete_s3_object
from app.utils.pdf import extract_text_from_pdf
from app.services.ai_service import AIService
import httpx

ALLOWED_CONTENT_TYPES = {'application/pdf', 'application/x-pdf'}
MAX_FILE_SIZE_BYTES = 5 * 1024 * 1024 # 5MB

class ResumeService:

    @staticmethod
    async def validate_file(file: UploadFile) -> bytes:
        if file.content_type not in ALLOWED_CONTENT_TYPES:
            raise HTTPException(400, detail={
                'error': 'Only PDF files are accepted',
                'code': 'INVALID_FILE_TYPE'
            })
        content = await file.read(MAX_FILE_SIZE_BYTES + 1)
        if len(content) > MAX_FILE_SIZE_BYTES:
            raise HTTPException(400, detail={
                'error': f'File size exceeds 5MB limit. Received: {len(content)/1024/1024:.1f}MB',
                'code': 'FILE_TOO_LARGE'
            })
        return content

    @staticmethod
    async def process_resume(
        db: AsyncSession, user_id: str,
        file_url: str, content: bytes,
        background_tasks: BackgroundTasks
    ) -> Resume:
        # Create or update resume record
        existing = await db.execute(select(Resume).where(Resume.user_id == user_id))
        resume = existing.scalar_one_or_none()

        if resume:
            # Delete old file from S3 before replacing
            background_tasks.add_task(delete_s3_object, resume.file_url)
            resume.file_url = file_url
            resume.parse_status = 'pending'
        else:
            resume = Resume(user_id=user_id, file_url=file_url, parse_status='pending')
```



```
        db.add(resume)

    await db.commit()
    await db.refresh(resume)

    # Trigger async AI parsing in background (non-blocking)
    background_tasks.add_task(
        ResumeService.run_ai_parsing, db, resume.id, content
    )
    return resume

    @staticmethod
    async def run_ai_parsing(db: AsyncSession, resume_id: str, content: bytes):
        raw_text = extract_text_from_pdf(content)
        ai_result = await AIService.analyze_resume(raw_text)

        resume = await db.get(Resume, resume_id)
        resume.raw_text = raw_text
        resume.parsed_skills = ai_result['skills']
        resume.parsed_experience = ai_result['experience']
        resume.parsed_education = ai_result['education']
        resume.parsed_projects = ai_result['projects']
        resume.parse_status = 'completed'
        await db.commit()
```

SECTION 6

Application Module Implementation

6. Application Module Implementation

6.1 Application Service

```
# app/services/application_service.py
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select, func
from fastapi import HTTPException, BackgroundTasks
from app.models.application import Application
from app.models.job import Job
from app.models.resume import Resume
from app.services.ai_service import AIService
from app.core.redis import redis_client

class ApplicationService:

    @staticmethod
    async def apply_for_job(
        db: AsyncSession, user_id: str, job_id: str,
        background_tasks: BackgroundTasks
    ) -> Application:

        # 1. Check job exists and is active
        job = await db.get(Job, job_id)
        if not job:
            raise HTTPException(404, detail={'error': 'Job not found', 'code': 'JOB_NOT_FOUND'})
        if job.status != 'active':
            raise HTTPException(400, detail={'error': 'This job is no longer accepting applications', 'code': 'JOB_CLOSED'})

        # 2. Check user has uploaded a resume
        resume_result = await db.execute(select(Resume).where(Resume.user_id == user_id))
        resume = resume_result.scalar_one_or_none()
        if not resume:
            raise HTTPException(400, detail={'error': 'Please upload your resume before applying', 'code': 'NO_RESUME'})

        # 3. Check for duplicate application (enforced at DB level too via UNIQUE constraint)
        dup_result = await db.execute(
            select(Application).where(
                Application.user_id == user_id,
                Application.job_id == job_id
            )
        )
```

```

    )
    if dup_result.scalar_one_or_none():
        raise HTTPException(409, detail={'error': 'You have already applied for
this job', 'code': 'DUPLICATE_APPLICATION'})

    # 4. Create application record
    application = Application(
        user_id=user_id, job_id=job_id,
        resume_url=resume.file_url, status='applied'
    )
    db.add(application)

    # 5. Increment job applicant counter
    job.applicant_count = (job.applicant_count or 0) + 1

    await db.commit()
    await db.refresh(application)

    # 6. Trigger AI scoring asynchronously (does not block response)
    background_tasks.add_task(
        ApplicationService.run_ai_scoring,
        db, application.id, resume.raw_text, job.description,
        job.required_skills
    )

    return application

    @staticmethod
    async def run_ai_scoring(
        db: AsyncSession, app_id: str,
        resume_text: str, job_desc: str, required_skills: list[str]
    ):
        try:
            result = await AIService.score_candidate(resume_text, job_desc)
            application = await db.get(Application, app_id)
            application.ai_score = result['score']
            application.matched_skills = result['matched_skills']
            application.missing_skills = result['missing_skills']
            await db.commit()
            # Cache the result
            cache_key = f'ai_score:{application.user_id}:{application.job_id}'
            await redis_client.setex(cache_key, 3600, str(result['score']))
        except Exception as e:
            logger.error(f'AI scoring failed for application {app_id}: {e}')
            # Fail silently - application still exists, score will be null

```

SECTION 7

Middleware Chain Implementation

7. Middleware Chain Implementation

7.1 Logging Middleware

```
# app/middlewares/logging_middleware.py
import time, uuid, structlog
from fastapi import Request, Response
from starlette.middleware.base import BaseHTTPMiddleware

logger = structlog.get_logger()

class LoggingMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next) -> Response:
        request_id = str(uuid.uuid4())
        start_time = time.time()

        # Attach request_id to request state for downstream use
        request.state.request_id = request_id

        # Extract user info from token if present (best-effort, no exception on
        # failure)
        user_id, user_role = None, None
        auth_header = request.headers.get('Authorization', '')
        if auth_header.startswith('Bearer '):
            try:
                payload = jwt.decode(auth_header[7:], settings.JWT_SECRET_KEY,
                                     algorithms=[settings.JWT_ALGORITHM])
                user_id = payload.get('sub')
                user_role = payload.get('role')
            except Exception:
                pass

        response = await call_next(request)

        duration_ms = round((time.time() - start_time) * 1000, 2)

        logger.info('request',
                    request_id = request_id,
                    method     = request.method,
                    path        = str(request.url.path),
                    status_code = response.status_code,
                    duration_ms = duration_ms,
                    user_id     = user_id,
                    user_role   = user_role,
                    ip          = request.client.host,
```

```

        user_agent = request.headers.get('user-agent', ''),
    )

    response.headers['X-Request-ID'] = request_id
    response.headers['X-Duration-Ms'] = str(duration_ms)
    return response

```

7.2 Error Handling Middleware

```

# app/middlewares/error_handler.py
# Global exception handlers registered on the FastAPI app

from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
from fastapi.exceptions import RequestValidationError
from sqlalchemy.exc import IntegrityError

def register_exception_handlers(app: FastAPI):

    @app.exception_handler(RequestValidationError)
    async def validation_error_handler(request: Request, exc: RequestValidationError):
        # Format Pydantic validation errors into clean user-facing messages
        errors = []
        for error in exc.errors():
            field = '.'.join(str(loc) for loc in error['loc'] if loc != 'body')
            errors.append({'field': field, 'message': error['msg']})
        return JSONResponse(status_code=422, content={
            'error': 'Validation failed',
            'code': 'VALIDATION_ERROR',
            'details': errors
        })

    @app.exception_handler(IntegrityError)
    async def db_integrity_handler(request: Request, exc: IntegrityError):
        # Handle DB-level constraint violations (e.g. duplicate application
        UNIQUE)
        logger.error('Database integrity error', exc_info=exc,
                     request_id=getattr(request.state, 'request_id', None))
        return JSONResponse(status_code=409, content={
            'error': 'A conflict occurred. This record may already exist.',
            'code': 'INTEGRITY_ERROR'
        })

    @app.exception_handler(Exception)
    async def generic_error_handler(request: Request, exc: Exception):
        # Never expose internal error details to the client in production
        logger.error('Unhandled exception', exc_info=exc,
                     request_id=getattr(request.state, 'request_id', None))
        return JSONResponse(status_code=500, content={
            'error': 'An unexpected error occurred. Please try again later.',
            'code': 'INTERNAL_ERROR',
            'request_id': getattr(request.state, 'request_id', None)
        })

```

```
}}
```

7.3 RBAC Role Middleware

```
# app/middlewares/role_middleware.py
from functools import wraps
from fastapi import HTTPException, Depends
from app.core.security import get_current_user
from app.models.user import User

def require_role(*allowed_roles: str):
    def decorator(func):
        @wraps(func)
        async def wrapper(*args, current_user: User = Depends(get_current_user),
            **kwargs):
            if current_user.role not in allowed_roles:
                raise HTTPException(status_code=403, detail={
                    'error': 'Access denied. Insufficient role.',
                    'code': 'INSUFFICIENT_ROLE',
                    'required': list(allowed_roles),
                    'actual': current_user.role
                })
            return await func(*args, current_user=current_user, **kwargs)
        return wrapper
    return decorator

# Ownership verification helper – used within service layer
async def verify_resource_owner(
    resource_owner_id: str,
    current_user: User,
    resource_name: str = 'resource'
) -> None:
    if str(resource_owner_id) != str(current_user.id) and current_user.role !=
    'admin':
        raise HTTPException(status_code=403, detail={
            'error': f'You do not have permission to modify this {resource_name}',
            'code': 'NOT_RESOURCE_OWNER'
        })
    )
```

SECTION 8

AI Service Client & Caching

8. AI Service Client & Caching

8.1 AI Service HTTP Client

```
# app/services/ai_service.py - HTTP client to the AI microservice
import httpx, json
from app.core.config import settings
from app.core.redis import redis_client
import structlog

logger = structlog.get_logger()

class AIService:
    BASE_URL = settings.AI_SERVICE_URL
    TIMEOUT = settings.AI_SERVICE_TIMEOUT # 10 seconds

    @staticmethod
    async def analyze_resume(resume_text: str) -> dict:
        async with httpx.AsyncClient(timeout=AIService.TIMEOUT) as client:
            try:
                response = await client.post(
                    f'{AIService.BASE_URL}/analyze-resume',
                    json={'resume_text': resume_text}
                )
                response.raise_for_status()
                return response.json()
            except httpx.TimeoutException:
                logger.error('AI service timeout on analyze-resume')
                # Return empty result - do not fail the resume upload
                return {'skills': [], 'experience': None, 'education': None,
'projects': []}
            except httpx.HTTPError as e:
                logger.error(f'AI service error: {e}')
                return {'skills': [], 'experience': None, 'education': None,
'projects': []}

    @staticmethod
    async def score_candidate(
        resume_text: str, job_description: str,
        user_id: str = None, job_id: str = None
    ) -> dict:

        # Check Redis cache first
        if user_id and job_id:
            cache_key = f'ai_score:{user_id}:{job_id}'
```

```

        cached = await redis_client.get(cache_key)
        if cached: return json.loads(cached)

    async with httpx.AsyncClient(timeout=AIService.TIMEOUT) as client:
        try:
            response = await client.post(
                f'{AIService.BASE_URL}/score-candidate',
                json={'resume_text': resume_text, 'job_description':
job_description}
            )
            response.raise_for_status()
            result = response.json()

            # Cache the result for 1 hour
            if user_id and job_id:
                await redis_client.setex(cache_key, 3600, json.dumps(result))
            return result

        except (httpx.TimeoutException, httpx.HTTPError) as e:
            logger.error(f'AI scoring failed: {e}')
            return {'score': None, 'matched_skills': [], 'missing_skills': []}

    @staticmethod
    async def recommend_jobs(user_id: str, skills: list[str]) -> list:
        cache_key = f'job_recs:{user_id}'
        cached = await redis_client.get(cache_key)
        if cached: return json.loads(cached)

    async with httpx.AsyncClient(timeout=AIService.TIMEOUT) as client:
        try:
            response = await client.post(
                f'{AIService.BASE_URL}/recommend-jobs',
                json={'user_id': user_id, 'skills': skills}
            )
            response.raise_for_status()
            result = response.json()
            await redis_client.setex(cache_key, 1800, json.dumps(result))
            return result
        except Exception as e:
            logger.error(f'Job recommendation failed: {e}')
            return []

```


SECTION 9

Pagination, Utilities & Helpers

9. Pagination, Utilities & Helper Functions

9.1 Generic Pagination Helper

```
# app/utils/pagination.py
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select, func
from sqlalchemy.sql import Select
from typing import TypeVar, Generic, Type
from pydantic import BaseModel
import math

T = TypeVar('T')

async def paginate(
    db: AsyncSession,
    query: Select,
    page: int,
    limit: int,
    response_model: Type[T] = None
) -> dict:
    # Count total matching records (executes separate COUNT query)
    count_query = select(func.count()).select_from(query.subquery())
    total = (await db.execute(count_query)).scalar_one()

    # Apply offset and limit for current page
    offset = (page - 1) * limit
    result = await db.execute(query.offset(offset).limit(limit))
    items = result.scalars().all()

    pages = math.ceil(total / limit) if total > 0 else 1

    return {
        'data': items,
        'total': total,
        'page': page,
        'pages': pages,
        'has_next': page < pages
    }

# app/utils/pdf.py - PDF text extraction
import fitz # PyMuPDF

def extract_text_from_pdf(content: bytes) -> str:
    doc = fitz.open(stream=content, filetype='pdf')
```

```

text = []
for page in doc:
    text.append(page.get_text('text')) # Extract plain text
doc.close()
# Join pages with newline, strip excessive whitespace
raw = ' '.join(text)
lines = [line.strip() for line in raw.splitlines() if line.strip()]
return ' '.join(lines)

```

9.2 Redis Utilities

```

# app/core/redis.py - Redis client and helpers
from redis.asyncio import Redis, ConnectionPool
from app.core.config import settings

redis_client: Redis = None

async def init_redis():
    global redis_client
    pool = ConnectionPool.from_url(
        settings.REDIS_URL,
        max_connections=20,
        decode_responses=True # Return strings instead of bytes
    )
    redis_client = Redis(connection_pool=pool)
    # Test connection
    await redis_client.ping()

# Helper: delete all keys matching a pattern (cache invalidation)
async def delete_pattern(pattern: str) -> int:
    count = 0
    async for key in redis_client.scan_iter(match=pattern):
        await redis_client.delete(key)
        count += 1
    return count

# Helper: get or set cache with TTL
async def cache_or_fetch(key: str, ttl: int, fetch_fn):
    cached = await redis_client.get(key)
    if cached:
        return json.loads(cached)
    result = await fetch_fn()
    await redis_client.setex(key, ttl, json.dumps(result, default=str))
    return result

```

SECTION 10

Error Response Standards

10. Error Response Standards

10.1 Standard Error Envelope

All API error responses must follow this exact JSON structure. No raw Python exception messages must ever reach the client in production.

```
// Standard error response envelope
{
  'error':      string,      // Human-readable error message (user-facing)
  'code':       string,      // Machine-readable error code (client logic)
  'details':    object,      // Optional: field-level details (validation errors)
  'request_id': string      // Request ID for support/debugging reference
}

// Examples:

// 400 Bad Request - File validation failure
{ 'error': 'Only PDF files are accepted', 'code': 'INVALID_FILE_TYPE' }

// 401 Unauthorized - Missing or invalid JWT
{ 'error': 'Invalid or expired token', 'code': 'INVALID_TOKEN' }

// 403 Forbidden - Wrong role
{ 'error': 'Access denied. Insufficient role.',
  'code': 'INSUFFICIENT_ROLE', 'required': ['recruiter'], 'actual': 'job_seeker' }

// 404 Not Found
{ 'error': 'Job not found', 'code': 'JOB_NOT_FOUND' }

// 409 Conflict - Duplicate application
{ 'error': 'You have already applied for this job', 'code':
  'DUPLICATE_APPLICATION' }

// 422 Validation Error - Pydantic schema failure
{ 'error': 'Validation failed', 'code': 'VALIDATION_ERROR',
  'details': [{ 'field': 'salary_max', 'message': 'salary_max must be >=
salary_min' }] }

// 429 Rate Limited
{ 'error': 'Rate limit exceeded', 'code': 'RATE_LIMIT_EXCEEDED', 'retry_after': 60
}

// 500 Internal Server Error - Never expose stack trace
```

```
{ 'error': 'An unexpected error occurred. Please try again later.',  
  'code': 'INTERNAL_ERROR', 'request_id': 'req_abc123' }
```

10.2 HTTP Status Code Reference

Status Code	When to Use	Example Scenarios
200 OK	GET requests that return data successfully	GET /jobs, GET /users/profile
201 Created	POST requests that create a new resource	POST /jobs, POST /applications
204 No Content	DELETE requests that succeed with no body	DELETE /jobs/:id, DELETE /resumes/me
400 Bad Request	Invalid input, business rule violation	File too large, job is closed
401 Unauthorized	Missing, invalid, or expired JWT token	No Authorization header, bad token
403 Forbidden	Valid token but insufficient role or ownership	Job seeker on POST /jobs
404 Not Found	Requested resource does not exist	GET /jobs/:nonexistent-id
409 Conflict	Duplicate resource or constraint violation	Duplicate email, duplicate application
422 Unprocessable Entity	Pydantic schema validation failure	Missing required field, wrong type
429 Too Many Requests	Rate limit exceeded	> 100 auth requests/min from one IP
500 Internal Server Error	Unexpected server-side error	Database connection failure, unhandled exception

SECTION 11

Testing Strategy & Requirements

11. Testing Strategy & Requirements

11.1 Test Structure

```
# tests/ - pytest test suite structure
tests/
  conftest.py           # Shared fixtures: test DB, test client, mock user
  test_auth.py          # Auth routes: register, login, refresh, logout
  test_users.py         # Profile CRUD, avatar upload
  test_jobs.py          # Job CRUD, search, filter, pagination
  test_resumes.py       # Upload validation, S3 mock, AI trigger
  test_applications.py  # Apply, duplicate check, status update
  test_ai_routes.py     # AI endpoint contracts (mock AI service)
  test_recruiter.py     # Analytics, top candidates, ranking
  test_middleware.py    # Rate limiting, RBAC, error handling

# conftest.py - Key fixtures
import pytest
from httpx import AsyncClient, ASGITransport
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from app.main import app

@pytest.fixture(scope='function')
async def test_db():
    # Use in-memory SQLite for unit tests (or test PostgreSQL for integration)
    engine = create_async_engine('sqlite+aiosqlite:///memory:')
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    yield AsyncSessionLocal(bind=engine)
    await engine.dispose()

@pytest.fixture
async def client(test_db):
    async with AsyncClient(transport=ASGITransport(app=app),
        base_url='http://test') as ac:
        yield ac

@pytest.fixture
async def job_seeker_token(client):
    response = await client.post('/v1/auth/register', json={
        'name': 'Test User', 'email': 'seeker@test.com',
        'password': 'Test@1234', 'role': 'job_seeker'
    })
    return response.json()['access_token']
```

11.2 Critical Test Cases

File	Test Name	Assertion	Priority
test_auth.py	test_register_success	POST /auth/register with valid data returns 201 + access_token	REQUIRED
test_auth.py	test_register_duplicate_email	POST /auth/register with existing email returns 409	REQUIRED
test_auth.py	test_login_wrong_password	POST /auth/login with wrong password returns 401	REQUIRED
test_auth.py	test_protected_route_no_token	GET /jobs without Authorization header returns 401	REQUIRED
test_jobs.py	test_create_job_as_job_seeker	POST /jobs by job_seeker returns 403	REQUIRED
test_jobs.py	test_create_job_as_recruiter	POST /jobs by recruiter returns 201 with job data	REQUIRED
test_jobs.py	test_update_others_job	PUT /jobs/:id by non-owner recruiter returns 403	REQUIRED
test_jobs.py	test_job_search_by_keyword	GET /jobs?search=python returns relevant results	REQUIRED
test_jobs.py	test_pagination	GET /jobs?page=2&limit=5 returns non-overlapping results	REQUIRED
test_resumes.py	test_upload_non_pdf	POST /resumes/upload with .docx returns 400	REQUIRED
test_resumes.py	test_upload_oversized	POST /resumes/upload with 6MB file returns 400	REQUIRED
test_applications.py	test_apply_without_resume	POST /applications without resume returns 400	REQUIRED
test_applications.py	test_duplicate_application	POST /applications to same job twice returns 409	REQUIRED

test_applications.py	test_apply_to_closed_job	POST /applications to closed job returns 400	REQUIRED
test_applications.py	test_job_seeker_cannot_update_status	PATCH /applications/:id/status by job seeker returns 403	REQUIRED

11.3 Coverage Requirements

Minimum Coverage	70% line coverage required across all modules. CI pipeline fails if coverage drops below this threshold.
Auth Module	95% coverage required. Authentication bugs are P0 security issues — near-complete coverage mandatory.
RBAC & Permissions	100% coverage required on role checks and ownership verification. Every denied path must be tested.
AI Service Client	Mock the AI service with httpx.MockTransport. Test both success paths and timeout/error fallback paths.
File Upload	Use Python io.BytesIO to mock file uploads. Test valid PDF, invalid type, oversized file — all three cases.
Database Constraints	Test UNIQUE constraint violations (duplicate email, duplicate application) to verify 409 responses.

SECTION 12

Configuration & Environment Management

12. Configuration & Environment Management

12.1 Pydantic Settings

```
# app/core/config.py - Centralised settings via Pydantic BaseSettings
from pydantic_settings import BaseSettings, SettingsConfigDict
from typing import Literal

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file='.env', env_file_encoding='utf-8')

    # Application
    APP_ENV: Literal['development', 'staging', 'production'] = 'development'
    APP_HOST: str = '0.0.0.0'
    APP_PORT: int = 8000
    DEBUG: bool = False

    # Database
    DATABASE_URL: str
    DB_POOL_MIN: int = 5
    DB_POOL_MAX: int = 20

    # Redis
    REDIS_URL: str
    REDIS_TTL_DEFAULT: int = 300

    # JWT
    JWT_SECRET_KEY: str
    JWT_ALGORITHM: str = 'HS256'
    JWT_ACCESS_TOKEN_EXPIRE_MINUTES: int = 1440
    JWT_REFRESH_TOKEN_EXPIRE_DAYS: int = 30

    # AWS
    AWS_ACCESS_KEY_ID: str
    AWS_SECRET_ACCESS_KEY: str
    AWS_REGION: str = 'ap-south-1'
    S3_BUCKET_NAME: str
    S3_PREIGNED_URL_EXPIRY: int = 900

    # AI Service
    AI_SERVICE_URL: str = 'http://ai-service:8001'
    AI_SERVICE_TIMEOUT: int = 10

    # Rate Limiting
    RATE_LIMIT_AUTH: int = 10
```



```

    RATE_LIMIT_GENERAL: int = 1000

    @property
    def is_production(self) -> bool:
        return self.APP_ENV == 'production'

settings = Settings() # Singleton - imported everywhere

```

12.2 Requirements File

```

# requirements.txt - Pinned production dependencies
fastapi==0.111.0
uvicorn[standard]==0.30.0
sqlalchemy[asyncio]==2.0.30
asynccpg==0.29.0
alembic==1.13.1
pydantic==2.7.1
pydantic-settings==2.3.0
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
boto3==1.34.107
redis[hiredis]==5.0.4
httpx==0.27.0
pymupdf==1.24.2
structlog==24.1.0
python-multipart==0.0.9

# Dev / test dependencies (requirements-dev.txt)
pytest==8.2.0
pytest-asyncio==0.23.6
pytest-cov==5.0.0
httpx==0.27.0
aiosqlite==0.20.0
ruff==0.4.4
safety==3.2.0

```

HireIQ BSD v1.0 — End of Document

This Backend Specification Document governs all API, service, database, and infrastructure implementation decisions for HireIQ v1.0.